Learned bias correction for surface-area estimation from CT-like voxel data

Manas Jain

Department of Physics, Indian Institute of Technology Roorkee, India

Abstract

Measuring the surface area of an object from a CT scan is harder than it looks, because the scan is a grid of small cubes (voxels) and the true surface passes between them. The two common methods are wrong in predictable ways. Counting the exposed faces of the voxels gives an area about 50% too large at every resolution. Marching cubes, which fits a triangle mesh to the data, is closer, but its error depends on how curved the object is and how noisy the scan is. We built 2,400 synthetic shapes from seven families whose true surface area is known to better than 0.01%, and turned them into 14,400 CT-like volumes at 32^3 and 64^3 voxels with three noise levels. We then trained several models to predict, for each object, how much the marching-cubes area should be corrected: a Random Forest on ten simple shape measurements, a 3D convolutional neural network (CNN) that looks at the whole volume, and a PointNet that looks at points on the surface. A short calculation based on blurring also predicts how much marching cubes underestimates the area of a sphere (-0.56% predicted against -0.62% measured at 32^3). On test shapes that none of the models had seen, marching cubes has an average error of 4.78% (32^3) and 3.52% (64^3). The best corrections bring this down to 0.68% (3D CNN, 32^3) and 0.18% (Random Forest, 64^3), and stay below 2% on two shape families left out of training altogether. The simple Random Forest does as well as the deep networks overall, the CNN is better only for objects smaller than 16 voxels, and PointNet is clearly weaker. No method can recover structures thinner than about two voxels.

1 Introduction

Surface area is one of the most requested measurements in X-ray computed tomography (CT) of materials and parts. It controls reaction rates in catalysts, flow and filtration behaviour in porous media, and the quality of additively manufactured lattices. In practice it is measured from a reconstructed voxel volume, either by counting the exposed faces of segmented voxels or by extracting a triangle mesh with marching cubes [1] and summing the triangle areas.

Both routes are biased. Counting voxel faces measures the area of a staircase, which is about 50% larger than the true surface at every resolution. Marching cubes on a thresholded (binary) volume is better, but it is a local method with a finite set of triangle orientations and its bias does not vanish as the voxel size shrinks [2, 3]. Marching cubes on the greyscale volume uses the partial-volume information and converges for clean, well-resolved data, but real CT volumes are noisy and many structures of interest are only a few voxels across. Noise roughens the extracted surface and adds small spurious components, while smoothing the volume to suppress noise shrinks curved surfaces and can erase thin features altogether.

The size and sign of the error therefore depend on resolution, noise, curvature and topology at the same time, which makes a single fixed correction factor inadequate. This is a natural setting for a learned correction: a model that looks at the volume, or at simple measurements computed from it, and predicts how far the classical measurement is from the truth for that particular object. Training such a model needs ground truth, which is only available exactly for synthetic objects.

This work builds a synthetic CT-like dataset of seven shape families with reference surface areas that are exact or converged to better than 10^{-4}, renders them at two resolutions and three noise levels, and compares four classical estimators, two simple calibrated corrections and two

learned models (a 3D convolutional network that sees the whole volume, and a PointNet that sees points on the marching-cubes surface). The models predict a multiplicative correction to the greyscale marching-cubes area, so that learning acts as a bias-correction layer on top of the classical measurement rather than replacing it.

2 Problem, objectives and motivation

Problem. Given a CT-like greyscale volume of a single object on an N^3 grid, estimate the area of the object's true, continuous surface. The volume is degraded by the finite voxel size (partial-volume averaging), by the system point-spread function and by noise.

Objectives.

1. Build a library of 3D shapes with reference surface areas accurate enough (relative error below 10^{-4}) that the errors of all estimators can be measured without contamination from the reference itself.
2. Quantify the bias and spread of the classical estimators (voxel-face counting and three marching-cubes variants) as a function of resolution and noise.
3. Train a 3D CNN and a PointNet to correct the greyscale marching-cubes area, and compare them, at matched resolution and noise, with the classical estimators and with two calibrated baselines: a per-regime lookup table and a Random Forest on ten simple measurements of the volume.
4. Test whether the learned corrections transfer to shape families never seen in training (superellipsoids with sharp edges or tips, and blobs with fine surface roughness).
5. State when a learned correction is worth using and when a classical estimator is enough.

Why this matters. A 5% error in surface area is large compared with the effects that surface-area measurements are often used to detect, for example differences between catalyst batches, changes in pore structure, or the deviation of a printed lattice from its design. Errors of this size are typical for marching cubes on binary data and for greyscale marching cubes on small or noisy objects (Section 6). A correction that is learned once on synthetic data and then applied per object is cheap at inference time and can be reused for any scanner whose resolution and noise level fall inside the training range.

3 Theory

This section explains why each classical method gets the area wrong, and by how much. In short: counting voxel faces measures a staircase instead of the real surface, so it is always too high; marching cubes on a black-and-white volume can only build a few triangle directions, so it leaves a small zig-zag that never goes away; and marching cubes on the greyscale volume is very good on flat surfaces but slightly too small on curved ones (because the scan blurs the surface) and too large on noisy ones. Knowing where each error comes from tells us what a correction model has to learn.

3.1 Voxel-face counting

Consider a planar piece of surface with unit normal $\mathbf{n} = (n_x, n_y, n_z)$. Its digital (staircase) version is made of voxel faces normal to the three axes, and the total area of the faces normal to axis i

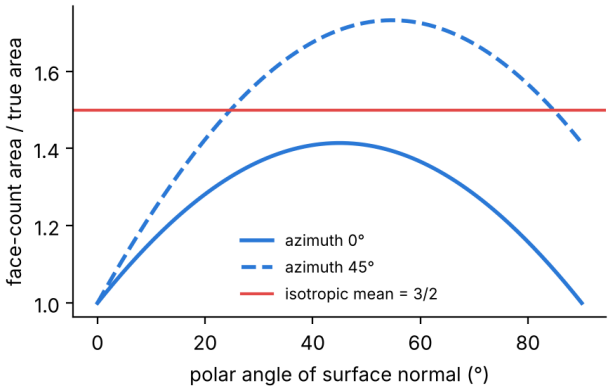


Figure 1: Face-count area per unit true area, $|n_x| + |n_y| + |n_z|$, for a plane whose normal makes the given polar angle with the z axis, at azimuth 0° and 45°. The isotropic average is 3/2.

equals the projection of the true area onto the plane perpendicular to that axis. The face-count area per unit true area is therefore

$$r(\mathbf{n}) = |n_x| + |n_y| + |n_z|, \qquad 1 \le r \le \sqrt{3}. \tag{1}$$

It equals 1 only for axis-aligned surfaces (Fig. 1). For normals distributed uniformly on the sphere each component is uniform on $[-1, 1]$, so $\mathbb{E}|n_i| = 1/2$ and $\mathbb{E}\, r = 3/2$. The face count of a randomly oriented smooth object thus overestimates its area by 50%, and since r does not depend on the voxel size, refining the grid does not help. On a clean sphere at 128^3 we measure +50.03%.

3.2 Marching cubes on binary and greyscale volumes

Marching cubes [1] places triangle vertices on the edges of each $2 \times 2 \times 2$ cell where the image crosses the iso-level. On a binary volume the crossing is always at the edge midpoint, so only a finite set of triangle orientations can occur, and the mesh approximates a plane of arbitrary orientation by a small-scale zig-zag whose excess area depends on the orientation but not on the voxel size. Using smaller voxels makes the zig-zag finer but not flatter, so the area error does not go to zero as the grid is refined [2, 3]. On clean spheres we measure +7.3% at 16^3, rising to +8.8% at 128^3.

On a greyscale volume the vertex position is interpolated from the partial-volume intensities, so the mesh can follow the surface between voxel centres. For a flat surface the half-intensity level of the blurred image lies exactly on the true surface. For a curved surface it does not: blurring spreads the object's intensity outwards over a larger volume, so the half-intensity surface ends up slightly inside the true one, and more so where the surface bends sharply. The result is an area that is a little too small. For a sphere of radius R voxels the expected error is about $-2\sigma^2/R^2$, where σ^2 measures the total blur ($\sigma^2 \approx 0.44$ voxel^2 in our data). This predicts −0.56% for a sphere on a 32^3 grid, and we measure −0.62%. The error shrinks quickly as objects get larger, but extra smoothing (often used to remove noise) makes it about three times bigger. The derivation and the full comparison are in Appendix A.

3.3 Effect of noise

Additive noise with standard deviation σ_n has two effects on marching cubes. First, it perturbs the iso-crossing positions near the surface. A randomly perturbed surface has more area than the smooth one, since the area element $\sqrt{1 + |\nabla\eta|^2} \ge 1$ for any height perturbation η, so this bias is always positive. Second, background voxels exceed the iso-level by chance with probability $p = Q(0.5/\sigma_n)$, where Q is the Gaussian tail function, and object voxels fall below it with the same probability. Each such voxel becomes a tiny closed surface or cavity. For $\sigma_n = 0.15$, $p = Q(3.33) \approx 4.3 \times 10^{-4}$, which predicts about 112 spurious components on a 64^3 grid and 14 on a

32^3 grid. These specks can be counted with the Euler number of the thresholded volume (roughly, the number of separate pieces and cavities minus the number of tunnels), since each speck adds one to it. The measured Euler numbers at this noise level are indeed about 100 to 125 at 64^3. For $\sigma_n = 0.05$, $p = Q(10)$ is negligible and only the surface roughening remains. Gaussian smoothing removes most of this noise, but as explained above it also shrinks curved surfaces, and it erases structures thinner than about $2\sigma_s$ entirely (we observe this for the thinnest tori on the 32^3 grid, where the smoothed volume never reaches the iso-level).

The upshot is that greyscale marching cubes has a negative, curvature-driven bias that grows at low resolution, plus a positive, noise-driven bias that grows with σ_n and with the number of voxels. The two can partly cancel, which is why a single correction factor cannot fix both.

3.4 Reference surface area

Measuring the error of each method needs the true area of every shape. Spheres, cylinders, tori and ellipsoids have exact formulas. Random blobs and superellipsoids do not, so their area is computed numerically: the surface is covered with a very fine triangle mesh at two levels of detail (about 82,000 and 328,000 triangles), and the two results are combined to cancel the leading error term. The remaining error is below 0.01% for every shape, far smaller than the errors being measured. The formulas are given in Appendix B.

3.5 Learning target

All models predict

$$y = \log \frac{A_{\text{true}}}{A_{\text{MC}}}, \tag{2}$$

where A_{MC} is the greyscale marching-cubes area, and the final estimate is $\hat{A} = A_{\text{MC}}\, e^{\hat{y}}$. The target is dimensionless, so it does not depend on the size of the object, and it is invariant to rotations and translations of the object. This lets the training data be augmented exactly with the 48 symmetries of the cube (axis permutations and reflections), which permute voxels without interpolation and leave the true area unchanged.

4 Previous work

Mesh-based measurement. Marching cubes [1] is the standard way to extract an iso-surface from a voxel volume; we use the topologically consistent variant of Lewiner et al. [4] as implemented in scikit-image [5]. Nyström et al. [6] compared areas of digital (voxel-face) surfaces and triangulated surfaces on phantoms and real objects and found the triangulated areas considerably more accurate, with errors decreasing as the voxel size decreased.

Local estimators and multigrid convergence. A large body of work in digital geometry estimates surface area from binary volumes by assigning a weight to each local $2 \times 2 \times 2$ configuration. Mullikin and Verbeek [7] derived optimal weights for digitized planes, and Windreich et al. [8] and Lindblad [9] optimised local configuration weights to minimise the bias for isotropically oriented surfaces. A central theoretical question is multigrid convergence, i.e. whether the estimate converges to the true area as the grid is refined. Kenmochi and Klette [2] showed that local polyhedrization schemes such as marching cubes on binary data are not multigrid convergent even for spheres, cubes and cylinders, and Svane [3] proved that no local digital algorithm for surface area is multigrid convergent on the class of convex polytopes. Ziegel and Kiderlen [10] studied design-based surface-area estimation from digitizations with random grid placement. Klette and Rosenfeld [11] give a general account. These results explain why the binary estimators in this work keep a resolution-independent bias, and they motivate using greyscale information.

Table 1: Shape families. Parameters are drawn uniformly from the given ranges; all shapes are normalised to maximum extent 1 before placement.

Family	Role	Shapes	Parameters
Sphere	train	150	radius 1
Ellipsoid	train	350	$a = 1$, $b, c \in [0.35, 1]$
Cylinder (capped)	train	300	radius $\in [0.3, 0.85]$, half-height $\sqrt{1-\rho^2}$
Torus	train	300	$r/R \in [0.2, 0.6]$, $R + r = 1$
Smooth blob	train	900	6 waves, $\omega \in [1, 3]$, s.d. of $\log r \in [0.15, 0.30]$
Superellipsoid	held out	200	$b, c \in [0.45, 1]$; $p \in [1.5, 1.8]$ (40%) or $[3, 6]$ (60%)
Rough blob	held out	200	smooth blob + 10 waves, $\omega \in [8, 14]$, s.d. $\in [0.02, 0.05]$

Learning-based estimation. Deep networks operating directly on voxel grids [12] and on unordered point sets [13] are established tools for 3D shape analysis. For CT of porous materials, Rabbani et al. [14] trained a convolutional network to predict a range of morphological, hydraulic, electrical and mechanical properties of porous samples from 3D images in a fraction of the time of image-based simulation. Random Forests [15] on hand-crafted descriptors remain a strong baseline for small tabular problems.

Gap. The work above either derives fixed estimators for binary data or learns global properties of complex microstructures, where an exact ground truth is not available. We did not find a study that measures how much a learned model reduces surface-area bias relative to marching cubes against an exact reference, separately for resolution, noise level and unseen shape families, and that compares deep models with a calibrated feature-based baseline. That comparison is the subject of this work.

5 Method

5.1 Shape families

Every shape is defined in a body frame with maximum extent 1 by an implicit function that is negative inside. Five families are used for training and two are held out for testing only (Table 1). The random blobs are star-shaped with

$$\log r(\mathbf{u}) = \sum_k a_k \cos(\omega_k \, \mathbf{d}_k \cdot \mathbf{u} + \phi_k) \,, \tag{3}$$

a sum of plane waves on the sphere with random unit directions $\mathbf{d}_k$ and phases ϕ_k, which behaves like a Gaussian random field on the sphere. Smooth blobs use six low-frequency waves; rough blobs add ten high-frequency, low-amplitude waves. Superellipsoids $|x/a|^p + |y/b|^p + |z/c|^p = 1$ are drawn either with $p < 2$ (pointed tips) or $p > 2$ (box-like, with rounded edges of high curvature), so they test both kinds of geometry the training families do not contain.

5.2 CT-like voxelization

Each shape is scaled so that its maximum extent is a fraction $f \sim \mathcal{U}[0.35, 1]$ of the half grid (minus a 2-voxel margin), rotated by a uniformly random rotation, and offset by a random sub-voxel shift. Object diameters range from 9.8 to 60 voxels. The volume is then formed by a simple CT forward model:

1. *Partial-volume occupancy.* Each voxel value is the fraction of the voxel inside the shape, computed with $5 \times 5 \times 5$ supersampling in a band of ± 4 voxels around the surface. The summed occupancy matches the true volume to within 10^{-4} for every family.

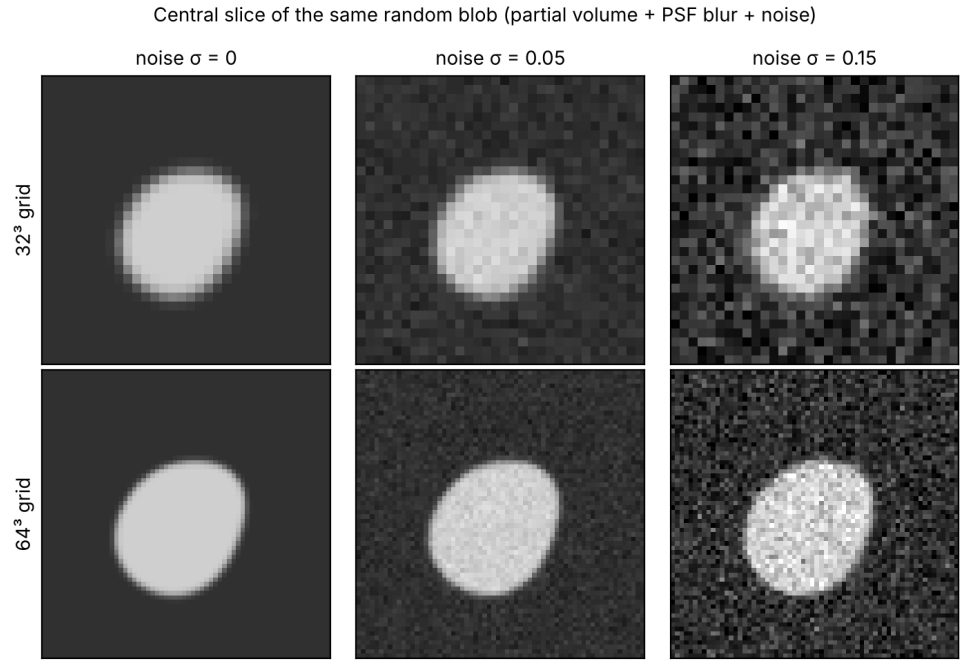


Figure 2: Central slice through the same random blob at both resolutions and all three noise levels.

2. *System blur.* A Gaussian point-spread function with $\sigma = 0.6$ voxel.
3. *Noise.* Additive Gaussian noise with $\sigma_n \in \{0, 0.05, 0.15\}$ relative to a unit object/background contrast.

Volumes are stored as 8-bit integers covering intensities $[-0.5, 1.5]$ (step 0.0078). Each shape is rendered at $N = 32$ and $N = 64$ with the same rotation and fill fraction, at all three noise levels, giving six volumes per shape and 14,400 volumes in total. Figure 2 shows an example.

The data are split by shape instance (70/15/15 within each training family), so all six variants of a shape fall into the same split and no information leaks between training and test. This gives 1,400 training, 299 validation and 301 test shapes from the training families, plus 200 superellipsoids and 200 rough blobs that are used only for testing.

5.3 Classical estimators

- **Voxel faces**: threshold at 0.5 and count exposed faces.
- **MC (binary)**: marching cubes on the thresholded volume.
- **MC (greyscale)**: marching cubes on the greyscale volume at iso-level 0.5.
- **MC (smoothed)**: Gaussian smoothing with $\sigma_s = 1$ voxel, then marching cubes at 0.5.

Figure 3 contrasts the binary and greyscale meshes.

5.4 Calibrated baselines

MC + lookup multiplies the greyscale MC area by $e^{\bar{y}}$, where $\bar{y}$ is the median training target for the same resolution and noise level. It is given the true noise level, which makes it optimistic; any learned model has to beat it to show that it does more than memorise one correction per regime. **Random Forest** [15] (400 trees, minimum leaf size 3, half of the features per split) predicts y from ten simple measurements of the volume: $\log(A_{\mathrm{MC}}/N^2)$, the log ratios of the face, binary-MC and smoothed-MC areas to A_{MC}, a compactness term $\log(V^{2/3}/A_{\mathrm{MC}})$ with V the summed intensity,

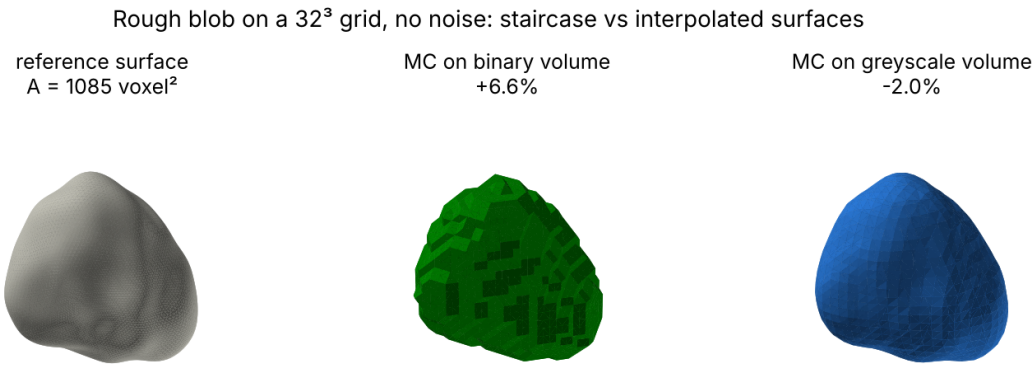


Figure 3: Reference surface of a rough blob and the marching-cubes meshes extracted from its clean 32^3 volume. The binary mesh shows the staircase pattern; the greyscale mesh is smooth but loses the finest surface detail.

two principal-axis aspect ratios, the Euler number of the thresholded volume, a background noise estimate, and N.

5.5 Learned models

3D CNN. Five convolutional blocks with 16, 32, 64, 128 and 256 channels. The first block has one 3^3 convolution, the others two, each followed by batch normalisation and ReLU, and each block ends with $2\times$ max pooling. Global average and max pooling feed a head of 128 hidden units with dropout 0.2 and one output (3,593,297 parameters). The input intensity is rescaled to $[-1, 1]$.

PointNet. 2,048 points are sampled uniformly by area on the greyscale marching-cubes mesh. Coordinates are centred on the grid centre and divided by a fixed constant (not per sample), so the point cloud keeps the voxel scale. A shared MLP of widths (64, 64, 64, 128, 1024), global max pooling and a head of 512 and 128 units with dropout 0.3 follow Qi et al. [13] (742,209 parameters). Instead of the T-Net alignment module we apply a uniformly random rotation to every training sample, which is exact because the target is rotation invariant.

Training. One model per resolution. Huber loss (threshold 1) on $y/0.05$, so that the loss is quadratic for errors below 5%; AdamW with weight decay 10^{-4}; one-cycle learning-rate schedule [16] with peak 10^{-3}; batch size 32; up to 40 epochs (15 for the CNN at 64^3, which costs about eight times more per epoch); early stopping after 10 epochs without improvement of the validation error, keeping the best checkpoint. The CNN is augmented with a random cube symmetry per batch. Training used PyTorch [17] with the Metal (MPS) backend on an Apple-silicon laptop; one epoch of the 32^3 CNN takes about 60 seconds.

5.6 Evaluation

For each estimate $\hat{A}$ we compute the relative error $e = \hat{A}/A_{\text{true}} - 1$ and report the mean absolute percentage error (MAPE), the bias (mean signed error), its standard deviation, the 95th percentile of $|e|$, and R^2 of $\hat{A}$ against A_{true}. All numbers are on held-out shapes: the test split of the training families and the two held-out families.

Table 2: Surface-area error (%) on held-out shapes, all noise levels pooled. MAPE: mean absolute error; bias: mean signed error. The two right-hand blocks are shape families never seen in training (MAPE). Best value per column in bold.

	Test MAPE		Test bias		Superellipsoid		Rough blob	
Method	32^3	64^3	32^3	64^3	32^3	64^3	32^3	64^3
Voxel faces	56.71	65.54	+56.64	+65.54	57.45	64.81	55.30	65.65
MC (binary)	10.38	14.49	+9.56	+14.49	10.51	14.31	8.91	13.99
MC (greyscale)	4.78	3.52	−1.25	+2.41	4.10	3.38	5.27	3.57
MC (smoothed)	13.78	3.04	−13.78	−3.04	9.93	2.29	14.02	4.37
MC + lookup	2.97	0.79	−1.45	−0.46	1.87	0.43	3.22	1.09
Random Forest	0.72	**0.18**	−0.18	**+0.04**	**0.73**	**0.39**	**1.71**	**0.48**
PointNet	1.98	1.04	+0.33	+0.34	1.54	0.95	3.00	1.32
3D CNN	**0.68**	0.26	**−0.10**	+0.10	1.05	0.45	1.96	0.55

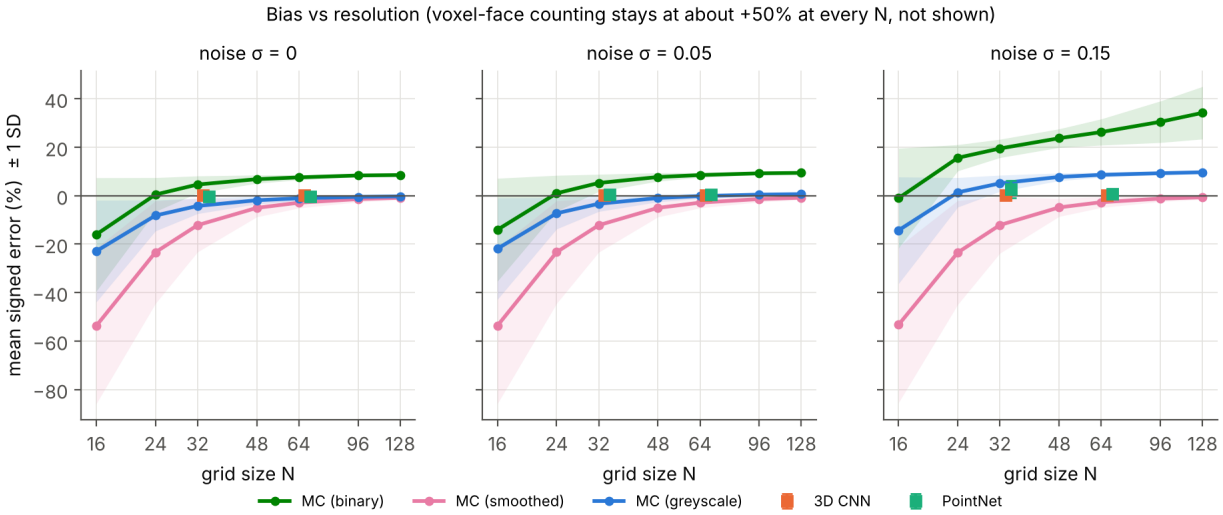


Figure 4: Mean signed error (±1 s.d.) of the marching-cubes estimators against grid size for 48 test shapes (8 per training family plus 8 superellipsoids), rendered with the same orientation and fill fraction at every N. Learned models are shown at the two sizes they were trained on.

6 Results

All results are on shapes that were not used for training or model selection: the test split of the five training families (301 shapes, 903 volumes per resolution) and the two held-out families (200 shapes each, 600 volumes per resolution). Table 2 gives the overall picture.

6.1 Classical estimators

Voxel-face counting overestimates the area by 57% at 32^3 and 66% at 64^3. This is more than the 50% predicted for clean, smooth surfaces because noise at $\sigma_n = 0.15$ adds isolated voxels whose faces also count; at that noise level the overestimate reaches 80% at 32^3 and 98% at 64^3 (Table 3).

Marching cubes on the binary volume has a positive bias (+9.6% at 32^3, +14.5% at 64^3) that grows with resolution instead of shrinking. The resolution sweep (Fig. 4) shows the same trend from 24^3 to 128^3: on clean data the bias rises from +0.5% to +8.5%, in line with the non-convergence results of Section 3; at $\sigma_n = 0.15$ it rises from +16% to +34%, because the number of spurious noise components grows with the number of voxels.

Greyscale marching cubes behaves as Section 3 predicts. Its bias is negative for small objects and low resolution (curvature shrinkage: −22.9% at 16^3 and −4.2% at 32^3 on clean data in the

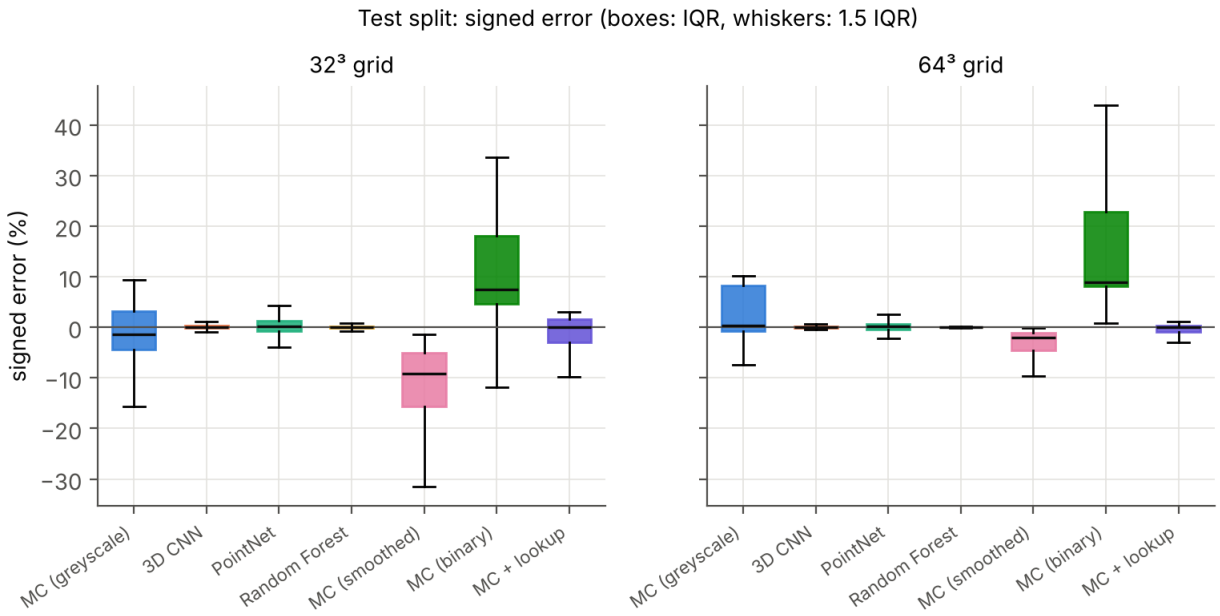


Figure 5: Distribution of the signed error on the test split. Boxes: interquartile range; whiskers: 1.5 IQR. Voxel-face counting is omitted for scale.

sweep) and becomes positive at high noise (+8.6% at 64^3 and +9.7% at 128^3 for $\sigma_n = 0.15$). Because the two effects have opposite signs, its error is not smallest on clean data: at 64^3 its test MAPE is 1.13% on clean data, 0.86% at $\sigma_n = 0.05$ and 8.56% at $\sigma_n = 0.15$. Smoothing before marching cubes removes most of the noise bias (2.93% MAPE at 64^3, $\sigma_n = 0.15$), but it costs 13.8% at 32^3, where most objects are small enough for the curvature shrinkage to dominate, and it erases thin tori completely (29.7% MAPE on tori at 32^3, Table 5).

6.2 Learned and calibrated corrections

Every correction model improves on raw greyscale marching cubes by a large factor (Table 2, Fig. 5). The simple lookup table, one factor per resolution and noise level, already reduces the test MAPE from 4.78% to 2.97% at 32^3 and from 3.52% to 0.79% at 64^3. That it works much better at 64^3 shows that most of the remaining error at 64^3 is a systematic, regime-dependent bias, while at 32^3 the error depends strongly on the individual object.

The 3D CNN reaches 0.68% at 32^3 and 0.26% at 64^3, a seven-fold and a thirteen-fold reduction relative to greyscale marching cubes, with a bias within ±0.1% and R^2 of 0.99985 and 0.99997. The Random Forest on ten simple measurements performs almost identically at 32^3 (0.72%) and is better at 64^3 (0.18%). PointNet is clearly weaker than both (1.98% and 1.04%): sampling 2,048 points from a mesh that already contains the noise artefacts throws away the intensity information that the volume-based models use.

The Random Forest's feature importances explain why a small model does so well. The most important measurements (impurity-based importance in brackets) are the Euler number (0.29), the noise estimate (0.26), the smoothed-to-raw marching-cubes area ratio (0.22) and the face-count-to-marching-cubes area ratio (0.08). The Euler number and the noise estimate measure exactly the two noise effects of Section 3 (spurious components and surface roughening), and the ratio of smoothed to raw marching-cubes area measures how much of the surface responds to smoothing, which is a proxy for curvature and thin features.

A failure case. The error distributions are heavy-tailed at 32^3: the standard deviation of the CNN error is 3.1% although its MAPE is 0.68%. Most of this comes from one test torus whose tube is 1.77 voxels thick. After the point-spread blur most of the tube stays below the iso-level,

Table 3: Test MAPE (%) by resolution and noise level. Best per column in bold.

	32^3			64^3		
Method	σ_n=0	0.05	0.15	0	0.05	0.15
Voxel faces	44.10	45.64	80.38	48.31	49.91	98.40
MC (binary)	5.24	5.85	20.05	7.64	8.49	27.34
MC (greyscale)	4.73	3.84	5.77	1.13	0.86	8.56
MC (smoothed)	13.87	13.82	13.65	3.11	3.09	2.93
MC + lookup	3.06	3.05	2.80	0.84	0.83	0.71
Random Forest	0.49	0.61	1.07	**0.09**	**0.14**	**0.30**
PointNet	1.42	1.41	3.13	0.75	0.69	1.67
3D CNN	**0.46**	**0.55**	**1.02**	0.20	0.19	0.40

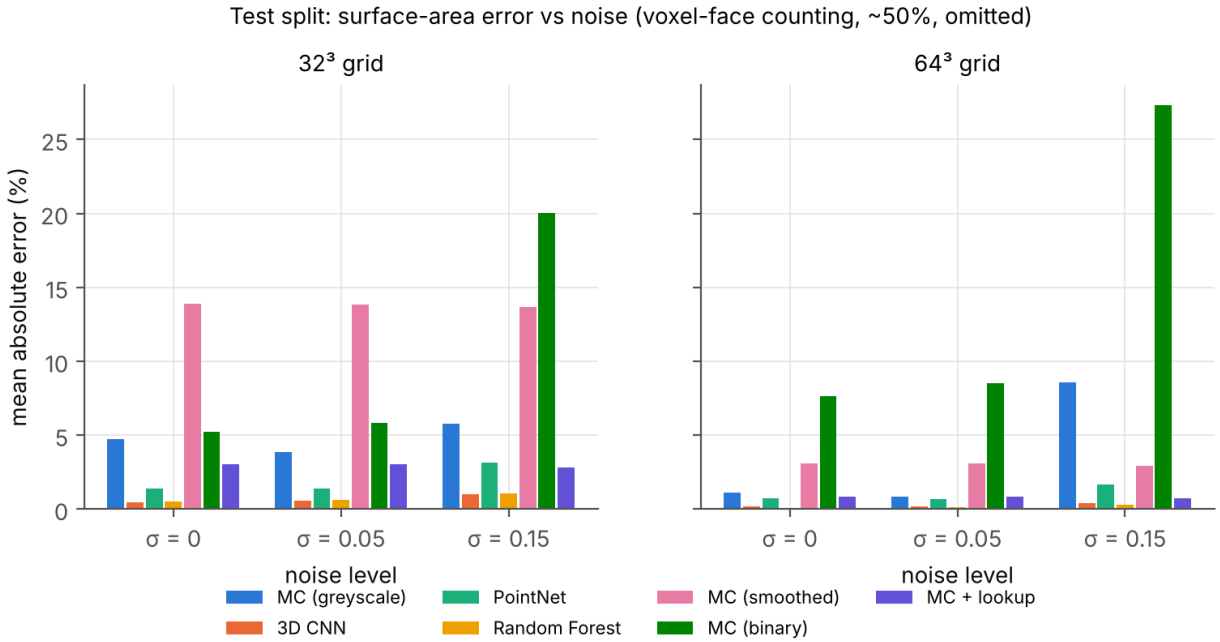


Figure 6: Test MAPE against noise level at both resolutions. Voxel-face counting (44–98%) is omitted.

so greyscale marching cubes recovers only 42 of its 142 voxel[2] (and the smoothed volume nothing at all). All correction models fail on it, with errors between −47% and −66% across models and noise levels. A multiplicative correction of the marching-cubes area cannot recover a surface that is mostly missing from the mesh; structures thinner than about two voxels need a different approach.

6.3 Noise and object size

Table 3 and Fig. 6 separate the noise levels. The learned models are nearly insensitive to noise: at 64^3 the CNN's MAPE goes from 0.20% (clean) to 0.40% ($\sigma_n = 0.15$), while greyscale marching cubes goes from 1.13% to 8.56%. The same holds at 32^3 (0.46% to 1.02%, against 4.73% to 5.77%).

Object size matters more than grid size (Table 4). For objects less than 16 voxels across, greyscale marching cubes has a MAPE of 6.43%, the Random Forest 1.41%, and the CNN 1.24%; this is the size class in which the CNN has its clearest advantage. For objects larger than 32 voxels the Random Forest is best, reaching 0.08% above 48 voxels, and even the lookup table stays below 0.7%.

Table 4: Test MAPE (%) by object diameter in voxels (both resolutions pooled).

Method	< 16	16–24	24–32	32–48	48–64
MC (greyscale)	6.43	3.89	3.57	3.45	3.44
MC (smoothed)	24.52	8.08	4.84	2.53	1.45
MC + lookup	4.76	1.78	1.48	0.67	0.58
Random Forest	1.41	**0.39**	**0.22**	**0.13**	**0.08**
PointNet	2.14	1.57	1.54	0.81	1.48
3D CNN	**1.24**	0.39	0.28	0.22	0.22

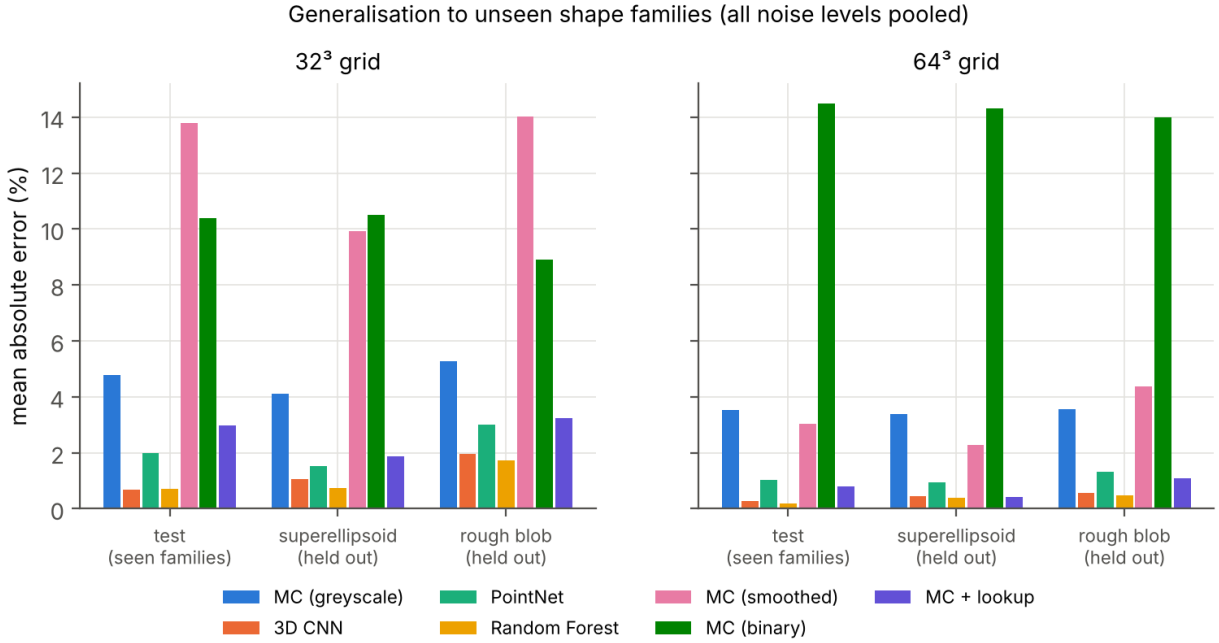


Figure 7: MAPE on the test split and on the two held-out families (all noise levels pooled).

6.4 Unseen shape families

Both held-out families stay far below the classical errors (Fig. 7, Table 2). On superellipsoids, whose sharp edges and pointed tips do not occur in the training data, the CNN reaches 1.05% and 0.45% and the Random Forest 0.73% and 0.39%, compared with 4.10% and 3.38% for greyscale marching cubes. These errors are between 1.0 and 2.2 times the test-split errors of the same models, so there is a measurable but moderate generalisation gap.

Rough blobs are the hardest case for every method: 1.96% (CNN) and 1.71% (Random Forest) at 32^3. Here the corrections are biased low (mean signed error -1.8% and -1.5%), as expected: the training shapes are smooth, so the models have never learned to add back area hidden in sub-voxel roughness. At 64^3, where more of the roughness is resolved, the error falls to about 0.5% and the bias vanishes. Table 5 breaks the results down by family.

6.5 Training behaviour

Figure 8 shows the validation error during training. The 32^3 CNN had nearly converged after 40 epochs. The 64^3 CNN, limited to 15 epochs by the compute budget, reached its best validation error in the last epoch, so its result is likely not the best this architecture can reach. PointNet's validation error flattens early at both resolutions.

Table 5: MAPE (%) by shape family (test split and held-out families, all noise levels). †Held out from training. Best per row and resolution in bold.

	32^3					64^3				
Family	MC grey	Lookup	RF	PointNet	3D CNN	MC grey	Lookup	RF	PointNet	3D CNN
Sphere	3.46	1.98	**0.19**	2.21	0.23	3.41	0.39	**0.06**	1.14	0.13
Ellipsoid	4.22	2.36	0.52	2.00	**0.45**	3.45	0.48	**0.20**	1.20	0.28
Cylinder	5.99	4.75	1.15	2.60	**0.78**	3.93	2.42	**0.31**	1.58	0.37
Torus	6.89	5.26	1.90	2.83	**1.85**	3.53	0.87	**0.26**	1.00	0.28
Smooth blob	4.12	2.03	**0.35**	1.45	0.41	3.43	0.41	**0.11**	0.79	0.23
Superellipsoid†	4.10	1.87	**0.73**	1.54	1.05	3.38	0.43	**0.39**	0.95	0.45
Rough blob†	5.27	3.22	**1.71**	3.00	1.96	3.57	1.09	**0.48**	1.32	0.55

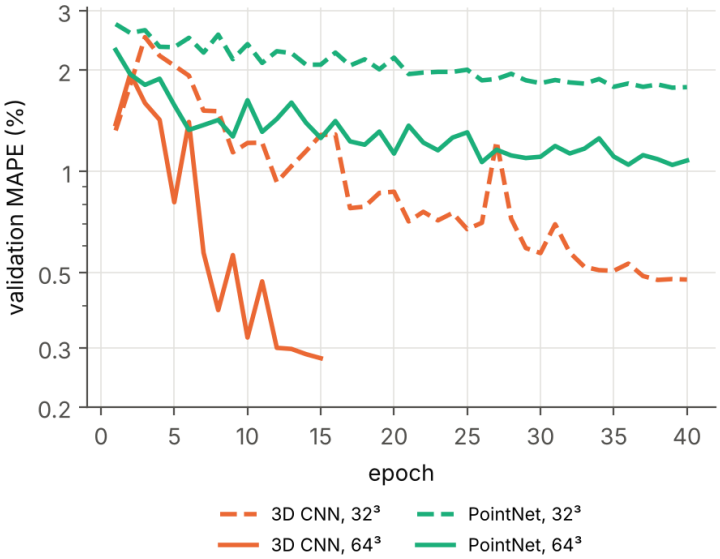


Figure 8: Validation MAPE during training (log scale).

6.6 Which estimator to use

Table 6 summarises when a learned correction is worth the effort. Its benefit is largest where the classical methods are weakest: small objects, coarse grids and noisy data. For large, clean objects greyscale marching cubes is already accurate to about 1%, and the gain from a learned model, while real, matters only when sub-percent accuracy is needed.

7 Conclusion

The errors of the classical surface-area methods are not random, and that is what makes them correctable. Counting voxel faces is always about 50% too high, because a staircase has more area than the smooth surface it follows. Marching cubes on a thresholded (black-and-white) volume keeps an error that does not shrink at higher resolution. Marching cubes on the greyscale volume is pulled down on curved surfaces, by an amount that a simple blurring argument predicts well, and pushed up by noise. Since both effects depend on the object and on the scan, no single correction factor can remove them.

A correction learned from synthetic shapes with known area removes most of the error. On test shapes it cuts the error of greyscale marching cubes from 4.78% to 0.68% at 32^3 and from 3.52% to 0.18% at 64^3. It barely changes with noise, and it still works, with errors below 2%, on shapes with sharp edges or rough surfaces that were never in the training data.

Table 6: Recommended estimator by regime, based on the test results.

Regime	Recommendation
Any noise level, any size	Do not use voxel-face counting or binary marching cubes for absolute area; their bias is large and does not decrease with resolution.
Clean data, large objects	Greyscale marching cubes is within about 1% at 64^3; a per-regime correction factor brings it below 1%.
Noisy data ($\sigma_n = 0.15$)	Raw marching cubes overestimates by up to 9–10%. Smoothing helps only for large objects; a learned correction (CNN or Random Forest) gives 0.3–1.1%.
Small objects (< 16 voxels)	Use the 3D CNN (1.24%, against 6.43% for marching cubes); smoothing is harmful.
Structures thinner than $\sim$2 voxels	No method tested here is reliable.

The most useful practical result is the comparison between models. A Random Forest on ten simple measurements of the volume does as well as a 3D CNN with 3.6 million parameters, and is the best model for large objects. Its two most useful measurements are the Euler number (roughly, the number of separate pieces and cavities minus the number of tunnels, which counts the noise specks) and an estimate of the noise level. The CNN is worth its extra cost mainly for small objects on coarse grids. PointNet, which only sees points on the noisy marching-cubes surface, is clearly worse than the models that see the whole volume.

Two cases are still not solved. Structures thinner than about two voxels disappear from the marching-cubes mesh before any correction can be applied, and roughness smaller than one voxel is simply not present in the image.

Natural next steps are a more realistic CT simulation (simulating the X-ray projections and the reconstruction, not only blur and noise), volumes with many touching structures such as foams and lattices, and a test on a real scan of an object whose surface area is known.

8 Limitations

- **Simplified physics.** The forward model is partial-volume averaging, a Gaussian PSF and white Gaussian noise added in the image domain. Real CT noise is correlated by the reconstruction filter and is signal dependent, and real volumes contain beam-hardening, ring and streak artefacts. A projection-domain simulation (forward projection, Poisson noise, filtered back-projection) would be the natural next step.

- **One isolated object per volume.** Porous media and lattices consist of many connected or touching structures, and the correction may depend on how much of the surface is close to other surfaces. The models have not been tested on such data.

- **Fixed acquisition parameters.** Blur, contrast and the iso-level are fixed, and the models are trained at two grid sizes and three noise levels. Their behaviour between and outside these settings is untested; in particular, a model trained at $\sigma_n \leq 0.15$ should not be trusted at higher noise.

- **Sub-voxel roughness is not observable.** Surface detail much smaller than a voxel does not survive voxelization. A learned model can only add back the excess area it expects from the roughness statistics of its training data, so its correction on rough surfaces is a prior, not a measurement, and it will be wrong if the real roughness spectrum differs.

- **Limited shape space.** The training shapes are star-shaped or of genus at most one. Shapes with many handles, thin sheets or concave pockets are not represented.

- **No tuning, single seed.** Architectures and hyperparameters were fixed in advance, the 64^3 CNN was trained for only 15 epochs because of the compute budget (a single laptop), and each model was trained once. Differences of a few hundredths of a percent between models should not be over-interpreted.
- **Real data.** No measurement on a real CT scan with a known surface area is included.

Code and data availability

All code, trained weights and result tables are in the project repository. The dataset is fully determined by the generation script and its fixed seed (51) and can be regenerated with one command; the generated volumes are also shared alongside the code.

A Curvature bias of greyscale marching cubes

For a plane and a symmetric blur the 0.5 iso-surface of the blurred image coincides with the true surface. For a curved surface it does not. If the effective blur is Gaussian with variance σ_{eff}^2, the level sets of the blurred indicator move towards the centre of curvature by

$$\delta \approx \sigma_{\text{eff}}^2 H, \qquad H = \tfrac{1}{2}(\kappa_1 + \kappa_2), \tag{4}$$

where H is the mean curvature. The intuition is that blurring a curved object spreads its intensity outwards over a larger volume, so the half-intensity surface ends up slightly inside the true one, and more so where the surface bends sharply. (Formally this follows from the heat equation, since Gaussian blurring is diffusion for a time $\sigma_{\text{eff}}^2/2$.) For a sphere of radius R this gives a relative area error of $-2\delta/R = -2\sigma_{\text{eff}}^2/R^2$. In our forward model the voxel box ($\sigma^2 = 1/12$) and the Gaussian PSF ($\sigma = 0.6$) combine to $\sigma_{\text{eff}}^2 \approx 0.443$ voxel^2; adding a smoothing filter with $\sigma_s = 1$ raises it to 1.443. Table 7 compares this prediction with the measured errors on clean spheres. The agreement is close at low resolution, where the curvature term dominates; at 64^3 the remaining discrepancy is of the same size as the interpolation error of marching cubes itself.

Table 7: Greyscale marching-cubes error on clean spheres: measured vs the curvature prediction $-2\sigma_{\text{eff}}^2/R^2$ from Eq. (4). R in voxels.

		MC on greyscale		MC after smoothing ($\sigma_s = 1$)	
Grid	R	predicted	measured	predicted	measured
16^3	5.4	−3.04%	−3.65%	−9.90%	−10.99%
32^3	12.6	−0.56%	−0.62%	−1.82%	−2.00%
64^3	27.0	−0.12%	−0.05%	−0.40%	−0.43%
128^3	55.8	−0.03%	+0.07%	−0.09%	−0.10%

B Reference surface areas

Spheres, cylinders and tori have closed-form areas. For an ellipsoid with semi-axes $a \geq b \geq c$,

$$A = 2\pi c^2 + \frac{2\pi ab}{\sin\varphi}\left[E(\varphi, m)\sin^2\varphi + F(\varphi, m)\cos^2\varphi\right], \quad \cos\varphi = \frac{c}{a}, \quad m = \frac{a^2(b^2 - c^2)}{b^2(a^2 - c^2)}, \tag{5}$$

with F and E the incomplete elliptic integrals of the first and second kind. The remaining families (random blobs and superellipsoids) are star-shaped, i.e. their surface is $\{r(\mathbf{u})\,\mathbf{u} : |\mathbf{u}| = 1\}$ for a radial function r. We triangulate them by mapping the vertices of a subdivided icosahedron to radius $r(\mathbf{u})$. For a smooth surface the area of such an inscribed mesh converges as $A_\ell =$

$A + c\,h_\ell^2 + o(h_\ell^2)$ in the edge length h_ℓ, and each subdivision halves h_ℓ, so Richardson extrapolation of two levels,

$$A_{\mathrm{ref}} = \tfrac{1}{3}\,(4A_{\ell+1} - A_\ell)\,, \tag{6}$$

cancels the leading error term. We use levels 6 and 7 (81,920 and 327,680 triangles) and keep $|A_{\mathrm{ref}} - A_7|$ as an error estimate for each shape.

References

[1] William E. Lorensen and Harvey E. Cline. Marching cubes: A high resolution 3D surface construction algorithm. *ACM SIGGRAPH Computer Graphics*, 21(4):163–169, 1987.

[2] Yukiko Kenmochi and Reinhard Klette. Surface area estimation for digitized regular solids. In *Vision Geometry IX, Proceedings of SPIE*, volume 4117, pages 100–111, 2000. doi: 10.1117/12.404839.

[3] Anne Marie Svane. On multigrid convergence of local algorithms for intrinsic volumes. arXiv preprint arXiv:1310.1783, 2013.

[4] Thomas Lewiner, Hélio Lopes, Antônio Wilson Vieira, and Geovan Tavares. Efficient implementation of marching cubes' cases with topological guarantees. *Journal of Graphics Tools*, 8(2):1–15, 2003.

[5] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D. Warner, Neil Yager, Emmanuelle Gouillart, and Tony Yu. scikit-image: image processing in Python. *PeerJ*, 2:e453, 2014.

[6] Ingela Nyström, Jayaram K. Udupa, George J. Grevera, and Bruce Elliot Hirsch. Area of and volume enclosed by digital and triangulated surfaces. In *Medical Imaging 2002: Visualization, Image-Guided Procedures, and Display, Proceedings of SPIE*, volume 4681, 2002. doi: 10.1117/12.466977.

[7] James C. Mullikin and Piet W. Verbeek. Surface area estimation of digitized planes. *Bioimaging*, 1(1):6–16, 1993.

[8] Guy Windreich, Nahum Kiryati, and Gabriele Lohmann. Voxel-based surface area estimation: from theory to practice. *Pattern Recognition*, 36(11):2531–2541, 2003. doi: 10.1016/S0031-3203(03)00173-0.

[9] Joakim Lindblad. Surface area estimation of digitized 3D objects using weighted local configurations. *Image and Vision Computing*, 23(2):111–122, 2005. doi: 10.1016/j.imavis.2004.06.012.

[10] Johanna Ziegel and Markus Kiderlen. Estimation of surface area and surface area measure of three-dimensional sets from digitizations. *Image and Vision Computing*, 28(1):64–77, 2010.

[11] Reinhard Klette and Azriel Rosenfeld. *Digital Geometry: Geometric Methods for Digital Picture Analysis*. Morgan Kaufmann, San Francisco, 2004.

[12] Daniel Maturana and Sebastian Scherer. VoxNet: A 3D convolutional neural network for real-time object recognition. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 922–928, 2015.

[13] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 652–660, 2017.

[14] Arash Rabbani, Masoud Babaei, Reza Shams, Ying Da Wang, and Traiwit Chung. DeePore: a deep learning workflow for rapid and comprehensive characterization of porous materials. *Advances in Water Resources*, page 103787, 2020.

[15] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[16] Leslie N. Smith and Nicholay Topin. Super-convergence: very fast training of neural networks using large learning rates. *Proceedings of SPIE, Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 11006, 2019.

[17] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.